STANFORD POWER CONTROL

Lab 1 | Genetic Algorithm and MOEA/D

Reproducible experimental report | MATLAB R2024b | 15 October 2026

Author / repository: Liu-Jiye / Lab1. All assessed deliverables are isolated in submission/. The original assignment and starter functions are retained separately in docs/ and src/.

Abstract

A real-coded GA was constructed from Sheffield GA Toolbox operators, and MOEA/D-DE was implemented directly. All 20 independent GA runs are strictly feasible after 50,000 candidate evaluations each. The median total power is 17.0013674875; the sample standard deviation is 2.65e-06. Three independent MOEA/D runs produce a median hypervolume of 73.83993948, using the prescribed minimization reference [26, 0]. The typical archive contains 5811 feasible, mutually nondominated power vectors. Saved outputs pass both MATLAB verification and independent NumPy objective, feasibility, nondomination and hypervolume checks.

The experiments use random initialization, fixed independent seeds, feasibility-first comparison and an objective-free segment repair. No known optimum, LP solution, CVX solution or independently optimized rate sample is injected into either evolutionary search. Independent LP and numerical hypervolume bounds are used only after experiments for validation.

1. Model and vectorized evaluators

There are five cochannel transmitter-receiver pairs. G(i,j) is the gain from transmitter j to receiver i; noise is 0.5 on every receiver. All powers satisfy 0.1 <= P_i <= 5. SINR values are linear, and rates are spectral efficiencies in bit/s/Hz under log2(1+SINR).

```
G = [1 .1 .2 .1 0;
     .1 1 .1 .1 0;
     .2 .1 2 .2 .2;
     .1 .1 .2 1 .1;
     0 0 .2 .1 1];
```

Define D = diag(diag(G)), H = G-D. For candidate P, SINR_i = G_ii P_i / (0.5 + sum_(j!=i) G_ij P_j), and c_i = 1/SINR_i - 0.5 <= 0. Task 1 minimizes f1 = sum_i P_i. Task 2 minimizes [f1, f2], where f2 = -sum_i log2(1+SINR_i). A lower f2 means higher sum-rate.

Interface	Input	Output
power_objective	N x 5 population	N x 1 total powers
power_constraints	N x 5 population	N x 5 inequalities; [] equalities; N x 5 SINR
evaluate_population	N x 5 population	N x 2 objectives, constraints, SINR, violation, feasible mask

The evaluators reject nonreal, nonfinite, nonpositive or incorrectly shaped inputs. Bounds are checked separately in evaluate_population. Matrix multiplication uses H transpose to respect receiver-row convention, even though the supplied G happens to be symmetric.

```
interference = X * H.' + 0.5;
sinrValue = (X .* diag(G).') ./ interference;
constraint = 1 ./ sinrValue - 0.5;
f1 = sum(X,2); f2 = -sum(log2(1+sinrValue),2);
```

2. Constraint handling and evaluation accounting

Strict feasibility requires all five c_i <= 0 and every bound satisfied. For MOEA/D, an inward c_i <= -1e-12 acceptance margin avoids scalar-versus-batch floating-point ambiguity at SINR=2. No 1e-4 relaxation is used, although the assignment rubric permits that violation threshold. Infeasible candidates are ranked by V = sum(max(c_i,0)) plus summed positive bound violations. Any feasible candidate outranks any infeasible one. Feasible GA candidates compare f1; feasible MOEA/D candidates compare their scalar subproblem values. Thus no penalty coefficient needs tuning.

2.1 Objective-free segment repair

For positive powers the SINR constraints are exactly linear: A P <= b, with A = 2H-D and b = -ones(5,1). Let p be a feasible parent and y a bound-clipped trial. For d = y-p, feasibility of p+alpha*d requires alpha <= (b_k-A_k p)/(A_k d) whenever A_k d > 0. The repair sets alpha to the minimum of these ratios and 1, clipped below by zero. If alpha < 1, multiply alpha by 0.98 to retain interior slack. Bounds remain satisfied by convexity.

Repair is enabled only after a naturally evaluated feasible candidate exists. The selected feasible parent is the anchor, or an existing feasible population member is selected uniformly when that parent is infeasible. Before then, feasibility-first evolution must find feasibility. Repair neither evaluates an objective nor solves an optimization problem. The 0.98 contraction balances numerical interior slack against access to constraint boundaries; shrinking variation still permits arbitrarily close approach to the optimum.

2.2 Budget and history conventions

One evaluation means one candidate power vector jointly evaluated for objectives and constraints. Initial candidates, all children, duplicates and repaired children count. Cached parent values are reused. GA uses 200 initial candidates plus 249 batches of 200 = 50,000. Post-search result recomputation and separate validation are not search evaluations. Every generation has a saved best feasible value; before feasibility this value is Inf. Medians include all 20 histories, preserving Inf rather than dropping failed runs. The first-feasible count records the actual individual index within the first successful batch.

3. Task 1: GA construction and parameter rationale

Parameter	Value and rationale
Representation / initialization	Five real powers; independent uniform [0.1,5] initialization.
Population / survival	200; (mu+lambda) elitist survival retains progress while evaluating exactly 200 children per generation.
Selection	Toolbox ranking with pressure 1.8 then SUS; stable selection pressure without objective-scale dependence.
Recombination	Toolbox extended intermediate recint; explicit probability 0.9. Its own crossover argument is ignored by the toolbox implementation.
Mutation	Toolbox mutbga, per-variable probability 0.2 = 1/5. Shrink = max(0.002,(1-(generation-1)/250)^2).
Constraint handling	Feasibility-first ordering, total positive violation, segment repair contraction 0.98.
Runs / budget	Seeds 11001 to 11020; 50,000 candidates per run including initialization.

Early variation explores broadly; the quadratic mutation shrink supports final precision while its nonzero floor prevents complete mutation freeze. Elitism protects feasible improvements. A separate pilot (seed 9101, excluded from statistics) confirmed these choices before the formal seeds were run. This is a reasoned parameter choice, not evidence of an exhaustive tuning study.

The driver directly calls ranking, sus, recint and mutbga from the external toolbox. It does not call MATLAB ga, gamultiobj, or any CVX optimizer. Tool source and the inaccessible course download are disclosed in Section 7.

4. GA results: 20 independent runs

Statistic (strictly feasible final runs only)	Measured value
Best	17.001364607366
Worst	17.001374901602
Mean	17.001367809850
Median	17.001367487497
Sample standard deviation (n-1)	2.65408670052e-06
Feasible-run rate	20 / 20 = 100%
Median first-feasible evaluations (all 20)	1619
|Median - 17.0014|	3.25125034628e-05 < 0.10

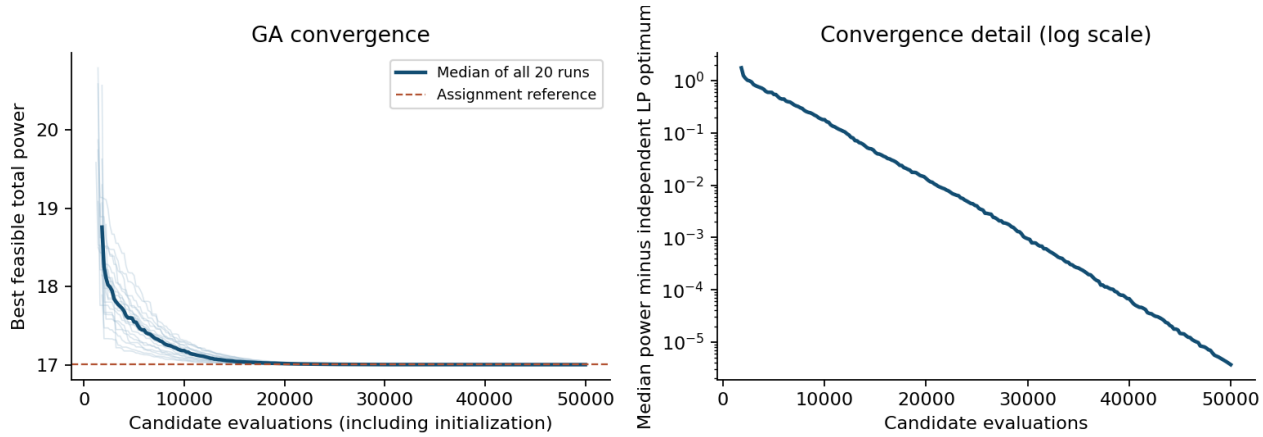


Figure 1. Every-generation best feasible histories (light), median across all 20 runs (dark), and reference power. Right: median optimality gap relative to a separate LP validation, never used by GA. Infinite pre-feasibility values are retained in data and omitted only from drawing finite coordinates.

All runs satisfy c_i <= 0 and SINR_i >= 2 on recomputation. The worst final objective is still within about 1.2e-5 of the independent LP minimum. The objective statistics use only the 20 feasible final solutions; no infeasible low-power candidate enters the table. The median difference from the assignment reference satisfies the highest numerical result bracket.

4.1 Per-run GA evidence

Run / seed	Final total power	First feasible eval.	max c_i
01 / 11001	17.0013684033	1605	-2.313e-09
02 / 11002	17.0013685580	1636	-2.721e-10
03 / 11003	17.0013651253	1966	-6.663e-10
04 / 11004	17.0013678658	1219	-1.196e-09
05 / 11005	17.0013673481	1363	-2.104e-09
06 / 11006	17.0013674297	2186	-5.150e-10
07 / 11007	17.0013648079	1604	-1.061e-10
08 / 11008	17.0013650215	1646	-8.831e-10
09 / 11009	17.0013688153	1079	-1.343e-09
10 / 11010	17.0013684443	1840	-1.384e-09
11 / 11011	17.0013646074	2978	-2.782e-10
12 / 11012	17.0013704540	1207	-1.346e-09
13 / 11013	17.0013675453	1633	-9.290e-10
14 / 11014	17.0013655673	1781	-6.780e-10
15 / 11015	17.0013716260	1217	-1.299e-09
16 / 11016	17.0013663128	1668	-1.045e-09
17 / 11017	17.0013749016	1352	-3.782e-09
18 / 11018	17.0013653486	1722	-3.955e-10
19 / 11019	17.0013669110	1464	-1.104e-09
20 / 11020	17.0013711038	1381	-3.647e-08

5. Task 2: self-implemented MOEA/D-DE

The algorithm decomposes the minimization problem into 201 scalar subproblems. Weights are w_i = [(i-1)/200, 1-(i-1)/200], including both endpoints. Each neighborhood contains the 20 closest weights by Euclidean distance (equivalently their first-coordinate distance). The ideal point z starts at the componentwise minimum of the evaluated initial population and is updated on every feasible child. Initially infeasible objective minima may remain as an optimistic lower ideal; no feasible comparison ever permits an infeasible replacement.

The scalar value for subproblem i is g_i(f) = max_j {max(w_ij,1e-6) * |f_j-z_j| / s_j}, with fixed scales s = [8,1.5]. These scales roughly match changes in total power and negative rate, preventing power magnitude from overwhelming the rate objective. The small weight floor keeps endpoint subproblems sensitive to both objectives.

Parameter	Value / reason
Population and weights	201 evenly spaced weights; fine front coverage with endpoints.
Generations / evaluations	1,000; 201 + 201*1,000 = 201,201 candidates per run.
Neighborhood / local mating	20; choose neighborhood with probability 0.9, otherwise all subproblems.
Differential variation	DE/rand/1/bin; F = 0.5; CR = 0.9; three distinct sampled parents; at least one mutant coordinate.
Polynomial mutation	Per-variable probability 0.2, distribution index eta = 20; bounded to [0.1,5].
Replacement	Feasibility-first; scalar comparison for feasible pairs; total violation for infeasible pairs; at most 2 replacements per child.
External archive	All evaluated strictly feasible nondominated points, deduplicated in objective space; no archive-size truncation.
Formal independence	Seeds 21001, 21002, 21003; entirely separate initialization and RNG state.

Local mating promotes neighboring trade-offs; occasional global mating reduces neighborhood isolation. Limiting replacement to two avoids a single child homogenizing the population. A pilot with 101 weights and 150 generations (seed 9201) checked feasibility and HV behavior. Formal settings increase front density and allow convergence; no pilot output enters the reported archives.

5.1 Main loop and archive

For each generation, visit subproblems in random order. Choose the mating pool; form a DE child; bound it; apply polynomial mutation and optional segment repair; evaluate once; update the feasible ideal; visit a shuffled replacement pool and replace at most two solutions under feasibility-first scalar comparison. After the generation, merge all feasible children into the archive and retain only unique nondominated objectives, preserving the matching power rows. Archive entries are evaluated candidates, rather than artificial interpolations. Full commented MOEA/D source is reproduced in Appendix A.

5.2 Exact two-dimensional HV

Both objectives are minimized. With nondominated points sorted by increasing x=f1 and decreasing y=f2, reference r=[26,0], set y_0=0 and compute HV = sum_i (26-x_i)*(y_(i-1)-y_i). This measures the union of rectangles without double counting. Points outside the reference rectangle are filtered; an empty feasible archive has HV=0. A second implementation independently integrates horizontal strips and agrees within 1e-9.

6. MOEA/D results and front quality

Run	Seed	Hypervolume	Archive K	Evaluations
1	21001	73.8398250807	3964	201201
2	21002	73.8443390925	4155	201201
3	21003	73.8399394753	5811	201201

The median-HV run is run 3 (seed 21003); HV = 73.8399394753. It is selected by sorting all three final HV values and choosing the middle run, rather than the largest. Its MAT file contains paretoSet (5811 x 5) and paretoFront (5811 x 2), together with referencePoint, typicalIndex, typicalSeed and typicalHV.

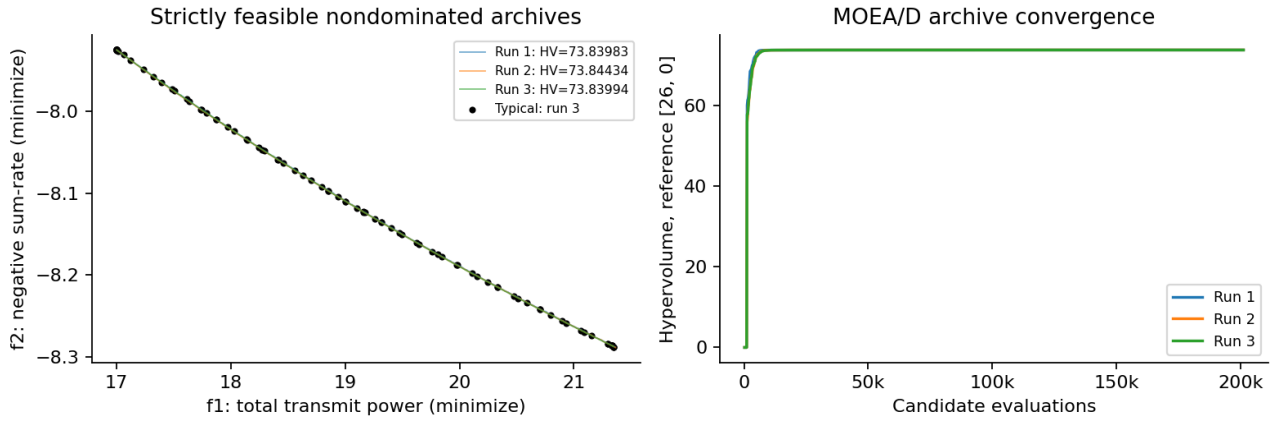


Figure 2. Objective axes follow the required minimization convention. All three archives are overlaid, with sampled markers identifying the typical run. HV history includes initialization and each complete generation.

Typical archive diagnostic	Value
Total-power range	17.00136379 to 21.34852250
Sum-rate range	7.92481250 to 8.28757208
Power coordinate range	2.9867030345 to 4.9973996379
Minimum SINR across every saved point/link	2.000000000004
Mutual nondomination / objective agreement	Independently verified for all points

The trade-off is physically consistent: higher feasible transmit power allows greater total rate, but interference and per-link QoS constraints limit the gain. All archives are mutually nondominated internally; finite evolutionary archives approximate the true Pareto set and do not prove global Pareto optimality of every point.

6.1 Evidence relative to the HV rubric

The instructor reference front is not supplied. A separate numerical upper-bound diagnostic gives HV_upper = 73.9627451153, and typical HV / HV_upper = 99.833963%. This strongly supports the rubric's 0.90 quality target for a valid feasible reference front, but is not a measured ratio against the unpublished instructor file and is not a guaranteed grade.

The bound uses an independent LP minimum-power value to establish the left integration limit. At each 0.05 power bin, LP bounds each interference over the feasible polytope with a power cap. The convex -log2(interference) term is upper bounded by its secant chord; log2(total received power) remains concave. A tangent affine function globally upper bounds that concave majorant, and an LP maximizes the tangent. Right-endpoint rectangles bound the integral of attainable rate versus power, including the final interval up to reference power 26. All calculations use floating-point tolerances and explicit padding, not interval arithmetic. These diagnostic samples never enter the GA or MOEA/D.

7. Verification, provenance and reproduction

MATLAB validate_implementation compares vectorized results with an independent nested-loop oracle on 37 random candidates, checks output dimensions and malformed input handling, verifies HV=36 for the hand case [(2,-3),(4,-5)] with reference [10,0], and checks empty, duplicate and dominated-point handling. It also verifies 1,000 segment repairs from a test-only feasible fixture. The fixture is never supplied to the experimental optimizers. Maximum evaluator oracle error is 8.88e-16.

verify_results reloads every saved run and checks strict feasibility, budgets, objective consistency, monotone best-so-far/HV histories, nondomination and median-run selection. independent_audit.py repeats objective/SINR calculations in NumPy, uses pairwise domination tests rather than the MATLAB scan, and a different HV strip formula. Both audit JSON files report passed=true. An initial boundary-roundoff audit failure was resolved by adding the inward 1e-12 margin and repeating the three MOEA/D runs with unchanged seeds and budgets; only these final validated runs are submitted. All 20 GA runs and all three MOEA/D archives are covered.

7.1 Toolbox and source disclosure

The course SharePoint GA Toolbox links returned HTTP 403 in this environment. The experiment therefore uses the public Sheffield Genetic Algorithm Toolbox mirror https://github.com/sumatrae/gatbx, commit 0cd0a5612edd6b0d5d97214927d1452fbac46986. Its identity with the course download has not been verified. MATLAB required changing external .M extensions to .m; operator content was not modified. No unmodified toolbox files are submitted, in accordance with the assignment. This provenance caveat must be checked if the course requires a specific distribution.

The GA driver, MOEA/D implementation, vectorized evaluators, repair, archives, validation and report were prepared for this submission with OpenAI Codex assistance. The original starter files are retained unchanged in src/. No Stanford CVX solver has been adapted as the GA solution. The independent SciPy LP/SLSQP diagnostic is isolated in validation/ and never called by the MATLAB optimizers.

7.2 Reproduction and files

```
cd('path/to/Lab1');
addpath('submission/matlab');
run_all('path/to/external/gatbx');
% Verify saved data without rerunning searches:
verify_results();
```

Run Python independent_audit.py and hv_upper_bound.py under submission/validation/, then report/build_report.py to rebuild this document and figures. Python dependencies: numpy, scipy, matplotlib, reportlab; PyMuPDF is used for PDF visual inspection. The build script reads saved MAT/JSON evidence and does not run optimization. Full instructions and scoring-item mapping are in submission/README.md and validation/rubric-checklist.md.

Assessed deliverable	Location relative to submission/
Vectorized evaluators / commented algorithms	matlab/*.m
20 GA histories / outputs / statistics	results/ga_results.mat; ga_run_01..20.mat; CSV/JSON
3 MOEA/D archives / histories	results/moead_results.mat; moead_run_01..03.mat
Required typical Pareto set/front MAT	results/pareto_typical.mat
PDF report and figures	report/lab_report.pdf; *.png
Verification and provenance	validation/*

References

[1] Supplied Lab - Stanford Power Control (1).docx: tasks, data and marking rubric. [2] Sheffield Genetic Algorithm Toolbox, public mirror cited above; operator documentation in external source. [3] Q. Zhang and H. Li, MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition, IEEE TEC 11(6), 2007, DOI: 10.1109/TEVC.2007.892759. [4] K. Deb, An efficient constraint handling method for genetic algorithms, CMAME 186, 2000, DOI: 10.1016/S0045-7825(99)00389-8. [5] Stanford/CVX power-control example, background only: cvxr.com/cvx/examples/gp_tutorial/html/power_control.html.

Appendix A. Complete commented MOEA/D source

The following source is identical to matlab/run_moead.m, including the local bounded polynomial mutation function. Supporting objective, repair, nondomination and HV routines are submitted alongside it.

```
function result = run_moead(seed,options)
%RUN_MOEAD MOEA/D-DE with normalized Tchebycheff decomposition.
% Independent random initialization; neighborhoods of adjacent weight vectors;
% DE/rand/1/bin and polynomial mutation; feasibility-first local replacement;
% an exact feasible nondominated external archive retains evaluated candidates.
if nargin<2, options=struct(); end
if ~isfield(options,'population'), options.population=201; end
if ~isfield(options,'generations'), options.generations=800; end
if ~isfield(options,'neighborhood'), options.neighborhood=20; end
N=options.population; T=options.neighborhood; rng(seed,'twister'); started=tic;
W=[linspace(0,1,N).',linspace(1,0,N).'];
[~,B]=sort(abs(W(:,1)-W(:,1).'),2); B=B(:,1:T);
X=0.1+4.9*rand(N,5); [F,~,~,V,feas]=evaluate_population(X,1e-12);
z=min(F,[],1); archiveX=X(feas,:); archiveF=F(feas,:);
[archiveX,archiveF]=nondominated2(archiveX,archiveF);
hvHistory=zeros(options.generations+1,1); hvHistory(1)=hypervolume2(archiveF);
evals=N; first=find(feas,1); if isempty(first), first=Inf; end
% Fixed scales cover comparable changes in power and negative sum-rate.
scale=[8,1.5]; delta=0.9; differentialWeight=0.5; crossoverRate=0.9;
for generation=1:options.generations
    newX=zeros(N,5); newF=zeros(N,2); newFeas=false(N,1);
    for i=randperm(N)
        if rand<delta, pool=B(i,:); else, pool=1:N; end
        r=pool(randperm(numel(pool),3));
        mutant=X(r(1),:)+differentialWeight*(X(r(2),:)-X(r(3),:));
        mask=rand(1,5)<crossoverRate; mask(randi(5))=true;
        child=X(i,:); child(mask)=mutant(mask);
        child=min(5,max(0.1,child));
        child=polynomial_mutation(child,0.2,20);
        if feas(i)
            child=repair_segment(child,X(i,:));
        elseif any(feas)
            candidates=find(feas);
    child=repair_segment(child,X(candidates(randi(numel(candidates))),:));
        end
        [cf,~,~,cv,ok]=evaluate_population(child,1e-12); evals=evals+1;
        if ok && isinf(first), first=evals; end
        if ok, z=min(z,cf); end
        % Feasible solutions compare scalar subproblem values; infeasible
        % solutions compare total violation. At most two replacements.
        count=0;
        for j=pool(randperm(numel(pool)))
            weight=max(W(j,:),1e-6);
            gChild=max(weight.*abs(cf-z)./scale);
            gOld=max(weight.*abs(F(j,:)-z)./scale);
            better=(ok && ~feas(j)) || (~ok && ~feas(j) && cv<V(j)) || ...
```

Appendix A. MOEA/D source (continued)

```
046                 (ok && feas(j) && gChild<gOld);
047             if better
048                 X(j,:)=child; F(j,:)=cf; V(j)=cv; feas(j)=ok;
049                 count=count+1; if count>=2, break; end
050             end
051         end
052         newX(i,:)=child; newF(i,:)=cf; newFeas(i)=ok;
053     end
054     [archiveX,archiveF]=nondominated2([archiveX;newX(newFeas,:)],[archiveF;newF(newFeas,:)]);
055     hvHistory(generation+1)=hypervolume2(archiveF);
056     if mod(generation,100)==0
057         fprintf('MOEAD seed %d generation %d HV %.8f archive
    %d\n',seed,generation,hvHistory(generation+1),size(archiveF,1));
058     end
059 end
060 result=struct('seed',seed,'paretoSet',archiveX,'paretoFront',archiveF, ...
061     'hv',hypervolume2(archiveF),'hvHistory',hvHistory,'referencePoint',[26 0], ...
062     'evaluations',evals,'firstFeasibleEvaluations',first,'options',options, ...
063     'weights',W,'neighborhoods',B,'scales',scale,'seconds',toc(started));
064 end
065
066 function x=polynomial_mutation(x,probability,eta)
067 % Bounded Deb polynomial mutation with per-variable probability 1/d.
068 for j=1:5
069     if rand>=probability, continue; end
070     d1=(x(j)-0.1)/4.9; d2=(5-x(j))/4.9; u=rand; exponent=1/(eta+1);
071     if u<=0.5
072         change=(2*u+(1-2*u)*(1-d1)^(eta+1))^exponent-1;
073     else
074         change=1-(2*(1-u)+2*(u-0.5)*(1-d2)^(eta+1))^exponent;
075     end
076     x(j)=min(5,max(0.1,x(j)+4.9*change));
077 end
078 end
```